eURC

Point of Sale on Azure
Hands-on Project 02 — Dev, Test and Production with DevOps CI/CD (GitHub Actions)

Application, DevOps and Azure architecture · build kit · deployment, test and monitoring design

Author	M. Akharuzzaman
Course context	AZ-900 hands-on series · Azure for Students subscription
Application package	Point of Sale — React 19 front end, Node.js 22 / Express 5 API
Delivery team	2 developers · 1 tester (QA engineer)
Region	Canada Central (canadacentral)
Document	EURC-P02-RPT · version 1.1 · 23 October 2026

Status: build kit complete and verified · ready for first deployment

Companion files: eURC_P02_Application_Architecture_A3 · eURC_P02_DevOps_Architecture_A3 · eURC_P02_Azure_Architecture_A3 (.drawio + .pdf, 2 sheets each) · eurc-p02-cicd-kit.zip

Document control

Version	Date	Change
1.0	2026-10-23	First issue: environments, pipeline and sample API.
1.1	2026-10-23	Application package specified as a **Point of Sale in React and Node.js**; delivery team of **2 developers and 1 tester** (CODEOWNERS, reviews, QA sign-off, Playwright E2E); architecture split into three separate drawings: Application, DevOps, Azure.

Contents

1. Executive summary 3
2. Objectives, scope and deliverables 3
3. Application architecture 5
4. DevOps architecture 9
5. Azure infrastructure architecture 13
6. Deployment procedure 16
7. Verification performed before deployment 17
8. Testing in Azure (procedure and evidence) 18
9. Monitoring and operations 18
10. Cost 20
11. Risks, limitations and decisions 20
Appendix A. Repository inventory 21

1. Executive summary

Project 02 delivers **eURC POS**, a multi-store Point of Sale, as a SaaS application on Azure. Store managers create a store, set up products and staff; cashiers ring up sales on a touch-friendly register, take cash (with change) or card (terminal approval code only), and print receipts. Managers refund sales and read a daily report. The front end is a **React 19** single-page app; the back end is a **Node.js 22 / Express 5** API using Azure SQL, Blob Storage and Key Vault through a managed identity.

The application runs in three isolated environments — **dev**, **test** and **prod** — defined in Bicep, and is delivered by a GitHub Actions pipeline that builds **one package** (API + React build) and promotes it unchanged. The team is two developers and one tester: developers review each other's pull requests, the tester owns the automated test suites and gives the **QA sign-off** that releases to production, and no one can approve their own change or their own production run.

Before a change reaches customers it passes: lint, 31 API tests and 17 React tests with 80 % coverage gates; a code-owner review; a smoke suite in dev; integration, load and Playwright browser tests in test; the tester's approval; a smoke test on the production staging slot; a zero-downtime slot swap; and a post-deployment gate that rolls back automatically on failure. No secrets exist anywhere: GitHub uses OpenID Connect and the app uses a managed identity.

3	2 + 1	10	≈ 4.90
architecture drawings: application, DevOps, Azure	developers + tester, with separation of duties	quality gates from pull request to production	USD per day with all environments running

> **Status.** The kit could not be run against your Azure subscription or GitHub organisation from where it was built, so the first live run is yours (section 6). Everything checkable offline was checked (section 7): all unit, component, integration, load and Playwright tests pass against the packaged build; Bicep, workflows and scripts lint cleanly; all 92 Azure CLI calls exist in the current CLI; and the pipeline and team-setup logic were rehearsed with simulated Azure and GitHub responses.

2. Objectives, scope and deliverables

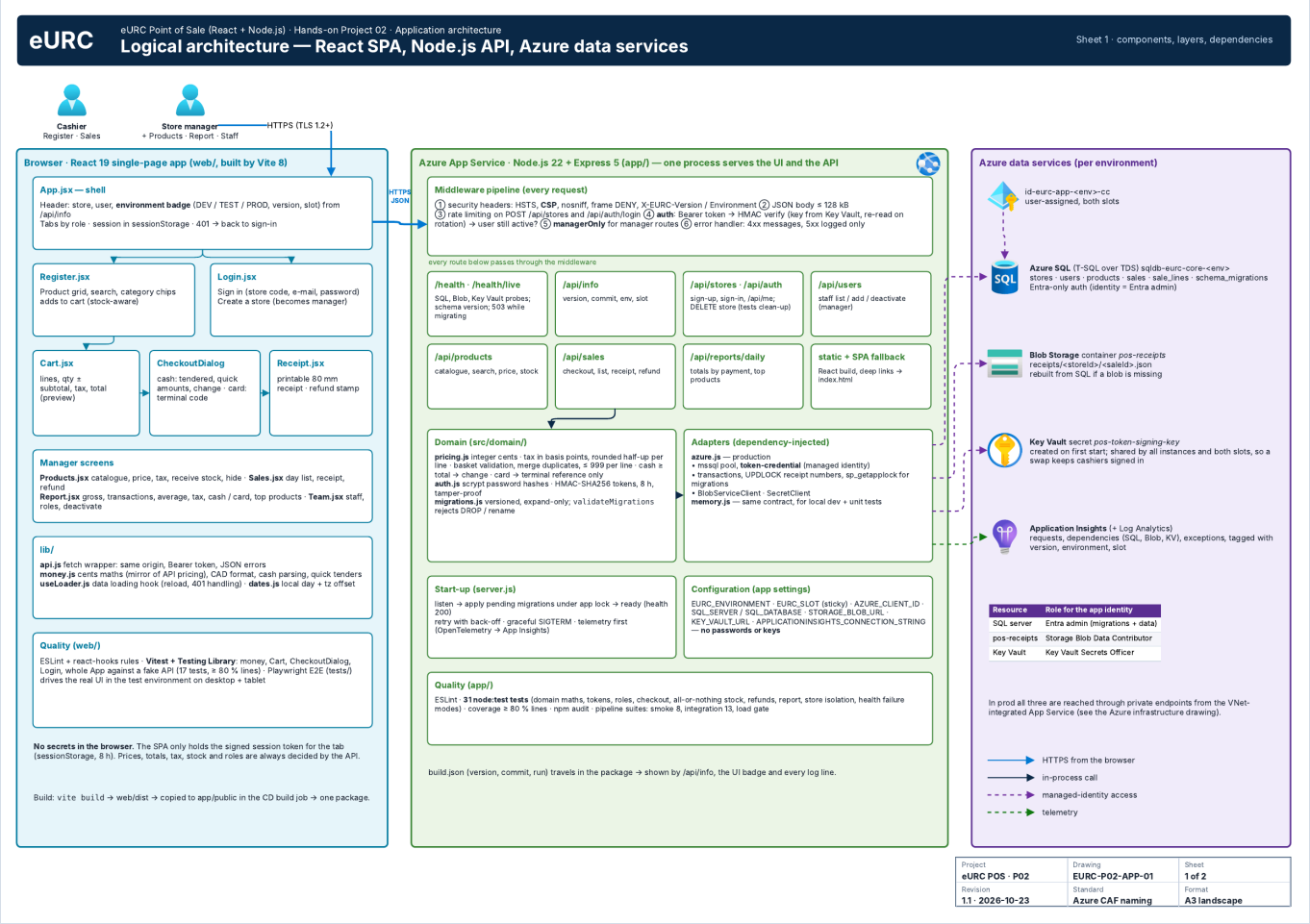
Requirement	How it is met	Where
Application package: Point of Sale in React and Node.js	React 19 SPA (register, payments, receipts, sales, products, report, staff) + Express 5 API (stores, roles, pricing, stock, refunds, reports)	web/, app/ · section 3
Team: 2 developers, 1 tester	Collaborators, CODEOWNERS, review ruleset, tester-owned tests/, QA sign-off on prod, RACI and Definition of Done	02-setup-github.sh, docs/TEAM.md · section 4
Dev, Test, Production	Three resource groups from one Bicep template with per-environment parameters	infra/ · section 5

Requirement	How it is met	Where
DevOps CI/CD	CI on every PR (app, web, infra, scripts); CD builds once and promotes with gates, approval, slot swap, rollback	.github/ · section 4
a) Architecture drawings	Three separate A3 drawings (draw.io + PDF), 2 sheets each, CAF names, title blocks	docs/eURC_P02_*_Architecture_A3
b) Configuration and installation scripts	setup.sh → 00 preflight, 01 Azure, 02 GitHub + team; 03 hardening	scripts/
c) Deployment, test and monitoring	Pipeline deploys and tests; 20-test-env.sh conformance; 30-monitor.sh operations and DORA metrics	scripts/20, 30 · sections 8–9

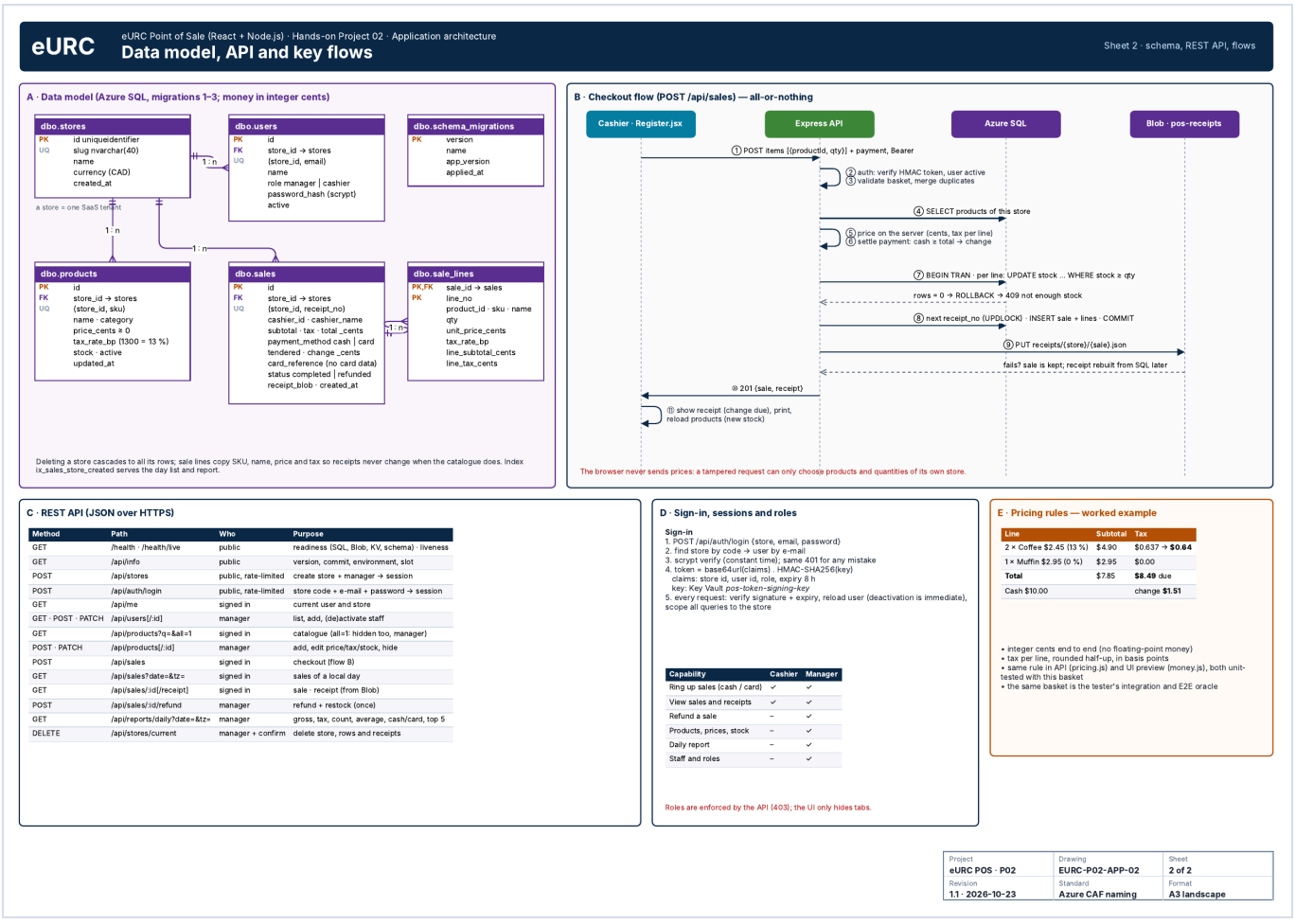
3. Application architecture

Drawing **eURC_P02_Application_Architecture_A3**. Sheet 1 shows the components and layers; sheet 2 the data model, the REST API and the checkout and sign-in flows.

Sheet APP-01 — Logical architecture



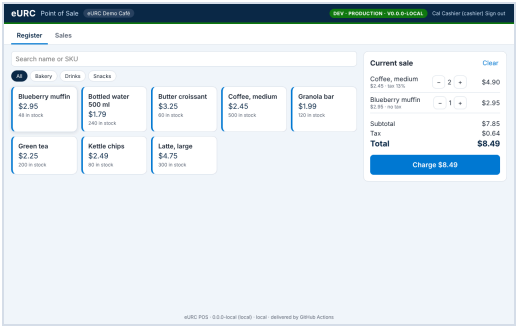
Sheet APP-02 — Data model, API and key flows



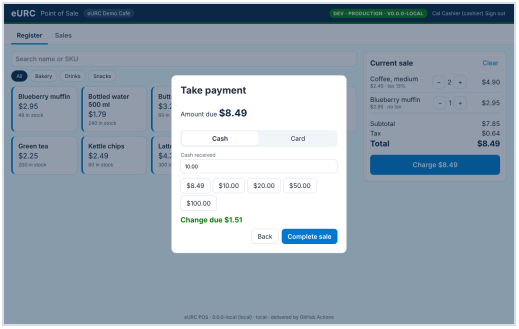
Reduced from A3; full-size editable versions are in docs/.

3.1 What the POS does

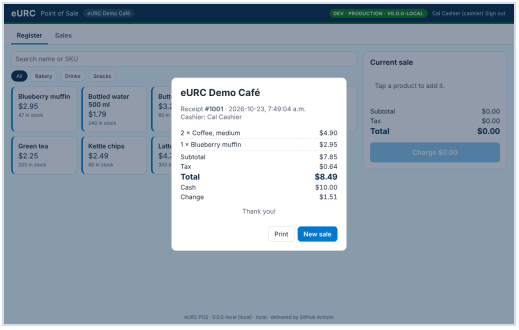
Screen (React)	Who	What it does	API
Sign in / Create a store	everyone	store code + e-mail + password; sign-up makes you the manager	/api/auth/login, /api/stores
Register	cashier, manager	product grid with search and categories, stock-aware cart, live totals	/api/products
Take payment	cashier, manager	cash with quick amounts and change; card with terminal approval code	/api/sales
Receipt	cashier, manager	80 mm printable receipt, stored as JSON in Blob Storage	/api/sales/:id/receipt
Sales	cashier, manager	sales of a day, open receipt; manager can refund (restocks)	/api/sales, .../refund
Products	manager	add products, price, tax %, receive stock, hide	/api/products
Report	manager	gross, transactions, average, tax, refunds, cash/card, top 5	/api/reports/daily
Staff	manager	add cashiers or managers, deactivate (takes effect immediately)	/api/users



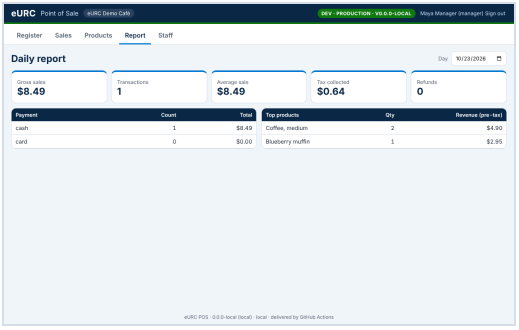
Register — cart with live totals (local build, DEV badge)



Take payment — cash, quick amounts, change due



Receipt — stored in Blob Storage, printable



Daily report (manager)

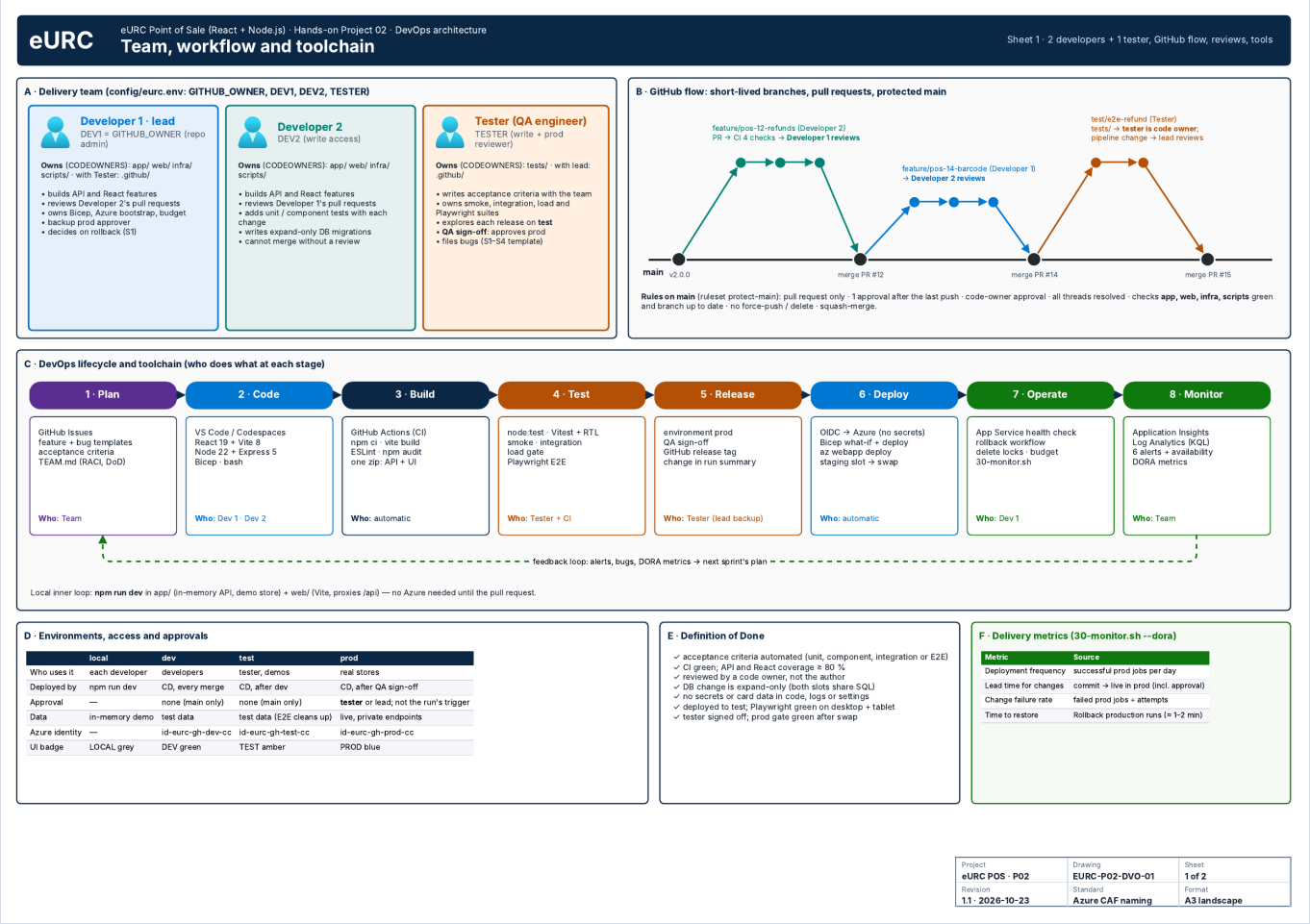
3.2 Design decisions

- **One deployable.** Express serves the React build and the API from the same App Service, so there is one origin (no CORS), one package to promote, and one slot swap per release.
- **Server-side money.** Integer cents, tax in basis points rounded half-up per line. The browser sends only product ids and quantities; prices, totals and change are computed by the API. The same rule is mirrored in the UI for the live preview and unit-tested on both sides with the same basket ($8.49, change $1.51).
- **All-or-nothing checkout.** Stock is decremented with `UPDATE … WHERE stock ≥ qty` for every line inside one SQL transaction; receipt numbers are allocated under an update lock. Overselling returns 409 and changes nothing.
- **Multi-store isolation.** Every query is scoped to the store in the signed session; a token from one store cannot read or sell another store's products.
- **Security.** scrypt password hashes; HMAC-SHA256 session tokens (8 h) signed with a key kept in Key Vault and shared by both slots; roles enforced by the API; CSP and security headers; rate-limited sign-in and sign-up; no card data is ever entered.
- **Testable by design.** Storage adapters are injected: Azure SQL / Blob / Key Vault in Azure, in-memory for `npm run dev` and unit tests.
- **Expand-only migrations.** Applied by the app at start-up under an SQL application lock; both slots share the database during a swap, so a unit test rejects DROP and rename.

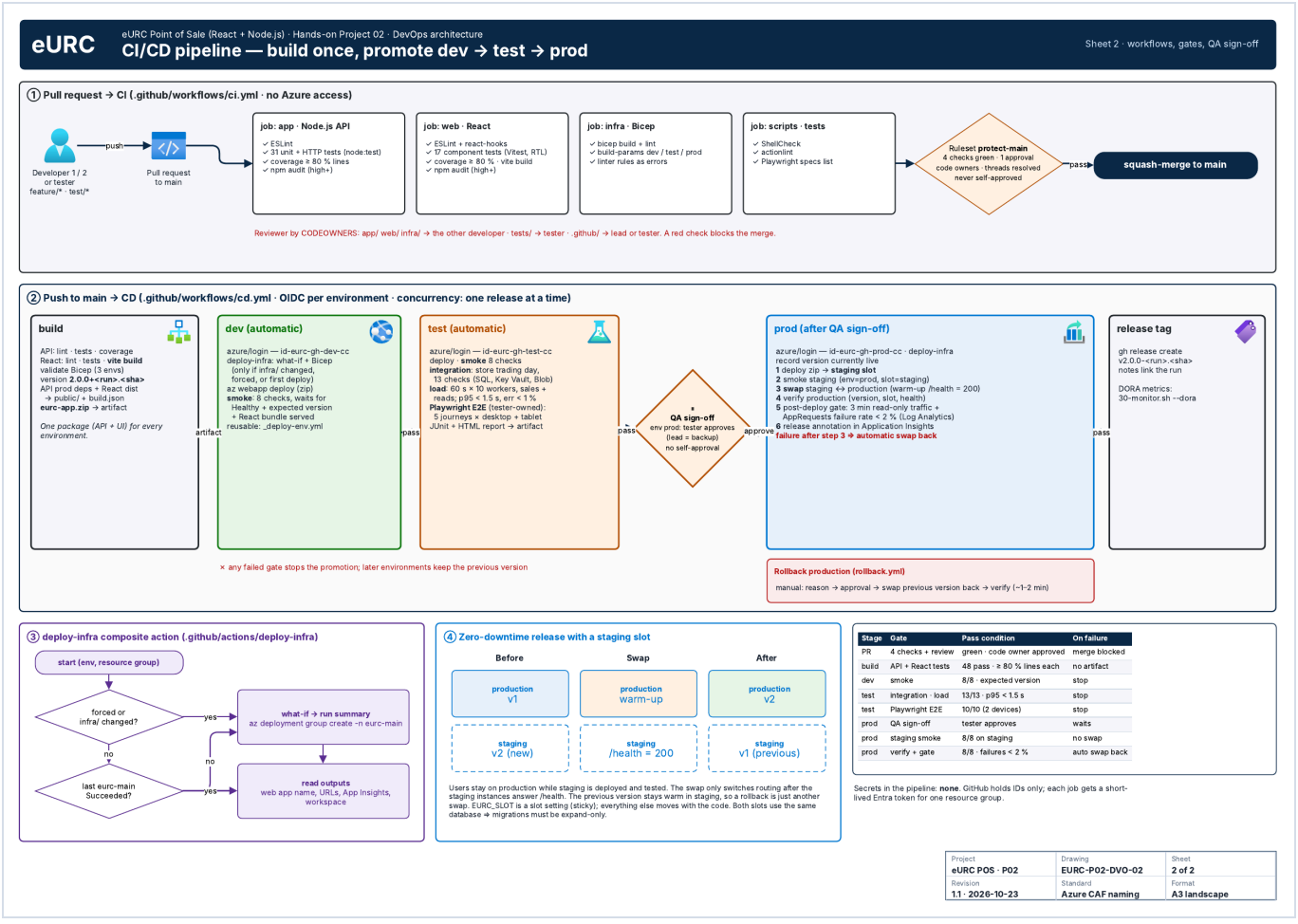
4. DevOps architecture

Drawing **eURC_P02_DevOps_Architecture_A3**. Sheet 1 shows the team, workflow and toolchain; sheet 2 the CI/CD pipeline and its gates.

Sheet DVO-01 — Team, workflow and toolchain



Sheet DVO-02 — CI/CD pipeline



4.1 Team and separation of duties

Activity	Developer 1 (lead)	Developer 2	Tester
React front end and Node.js API	R	R	C
Code review of app/, web/, infra/	reviews Dev 2	reviews Dev 1	C
Test suites in tests/ (smoke, integration, load, Playwright)	C	C	A / R (code owner)
Pipeline in .github/	A / R	C	R (co-owner)
Exploratory testing on test	I	I	R
QA sign-off → production	backup approver	—	A / R
Rollback decision	A	C	R (raises S1)

Configured by `02-setup-github.sh` from `config/eurc.env` (GITHUB_OWNER, DEV1, DEV2, TESTER): the developers and tester are invited with write access; `CODEOWNERS` is generated; the ruleset on main requires a pull request, one approval after the last push from a code owner of the changed area, resolved threads and the four green checks; the `prod` environment requires the tester (or the lead) and blocks self-approval. A developer therefore cannot weaken a test (tests/ is the tester's), ship without the other developer's review, or approve their own release.

4.2 Pipeline

Stage	What happens	Gate
PR (CI)	app: ESLint, 31 tests, coverage ≥ 80 %, npm audit · web: ESLint, 17 tests, coverage ≥ 80 %, vite build, audit · infra: Bicep build + lint · scripts: ShellCheck, actionlint, Playwright list	all 4 green + code-owner review
build	repeats the checks; version 2.0.0+<run>.<sha>; React build copied into the API's public/; one zip	tests, coverage
dev	OIDC login (dev identity); Bicep what-if + deploy if needed; zip deploy; smoke waits for Healthy + expected version + React bundle	smoke 8/8
test	same with the test identity; integration (a store's trading day), load (10 workers, 60 s, reads and sales), Playwright E2E on desktop and tablet	13/13 · p95 < 1.5 s · 10/10
QA sign-off	GitHub environment prod waits for the tester (lead as backup); trigger cannot approve	human
prod	staging slot deploy → smoke on staging → swap → verify → 3-min read-only traffic + failure rate from Log Analytics → release annotation	8/8 · failures < 2 %; else auto swap back
release	GitHub release named after the version	—

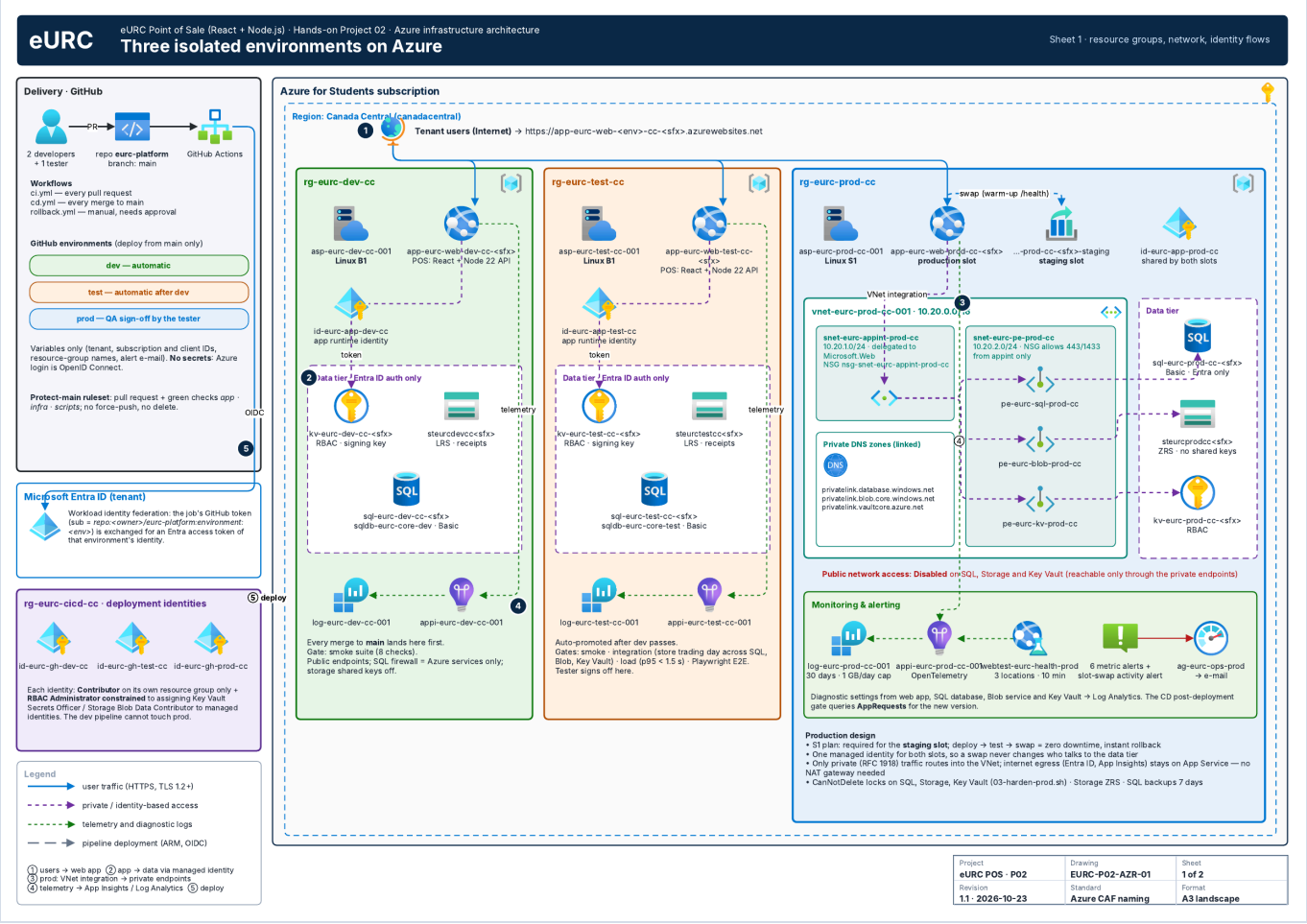
Build once: the zip tested in dev and test is byte-for-byte the one swapped into production. `build.json` inside it makes every environment report its version, commit and pipeline run (UI badge, `/api/info`,

telemetry). The prod gate sends read-only traffic so no test data is written to the live database. Rollback is a second slot swap (automatic on gate failure, or the *Rollback production* workflow, which needs the same approval).

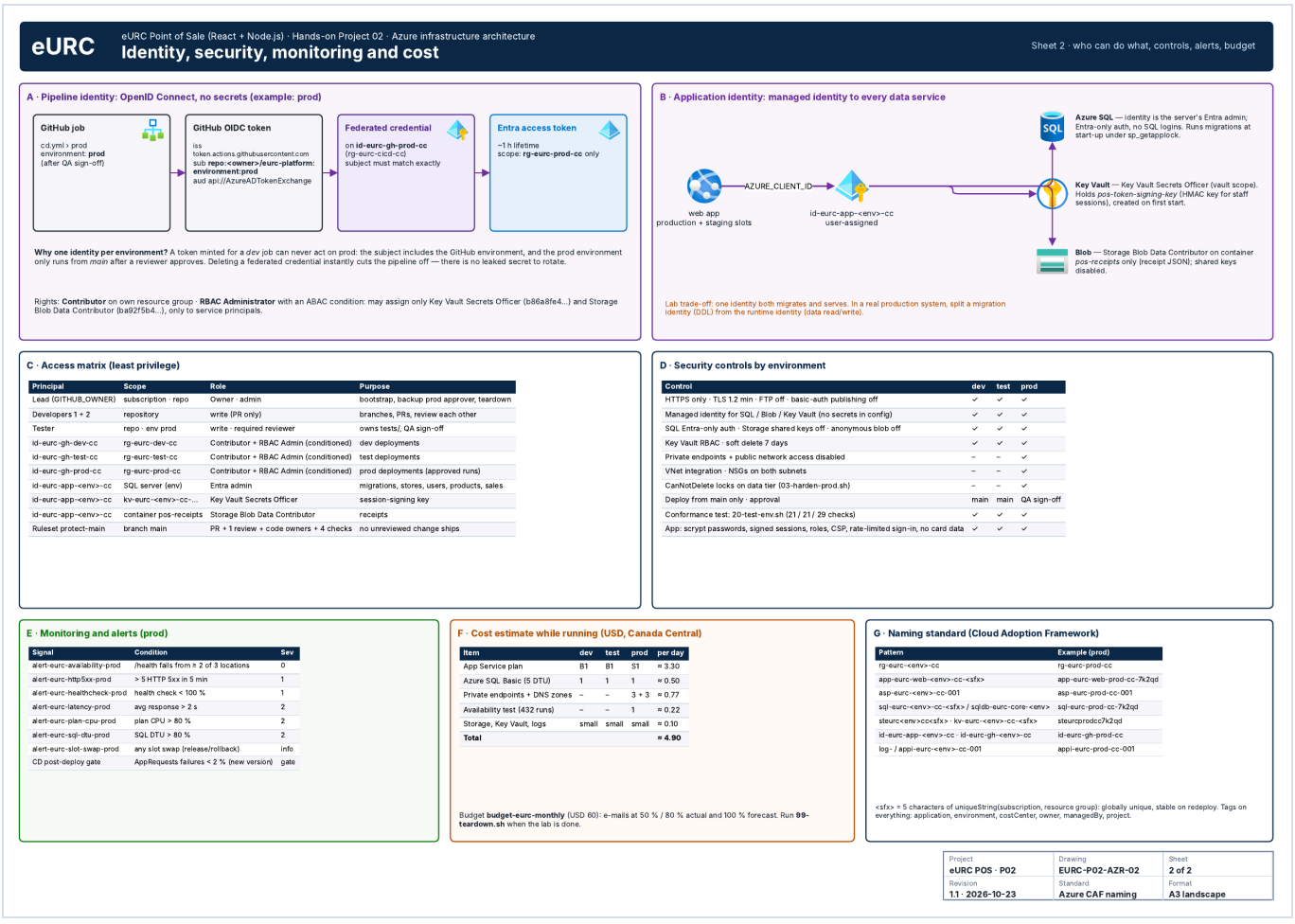
5. Azure infrastructure architecture

Drawing **eURC_P02_Azure_Architecture_A3**. Sheet 1: the three environments and networking; sheet 2: identity, security, monitoring and cost.

Sheet AZR-01 — Three isolated environments



Sheet AZR-02 — Identity, security, monitoring and cost



5.1 Environments

	dev	test	prod
Resource group	rg-eurc-dev-cc	rg-eurc-test-cc	rg-eurc-prod-cc
App Service plan	Linux B1	Linux B1	Linux S1 + staging slot
Azure SQL	Basic, Entra-only	same	Basic, Entra-only, private endpoint
Storage (pos-receipts)	LRS, shared keys off	same	ZRS, shared keys off, private endpoint
Key Vault	RBAC	RBAC	RBAC, private endpoint
Network	public endpoints	public endpoints	VNet 10.20.0.0/16, VNet integration, 3 private DNS zones, public access disabled
Monitoring	Log Analytics + App Insights	same	+ 6 metric alerts, availability test, slot-swap alert
Promotion	automatic	automatic after dev	QA sign-off by the tester

5.2 Infrastructure as code

`infra/main.bicep` with modules monitoring, network (prod), data, web and alerts (prod); the three environments differ only in `infra/env/*.bicepparam`. The pipeline deploys infrastructure only when `infra/` changed, when forced, or for a new environment, and always shows a what-if first. Names follow the Cloud Adoption Framework (`<type>-eurc-<env>-cc[-<sfx>]`); every resource is tagged.

5.3 Identity and security

- **Pipeline:** one user-assigned identity per environment (`id-eurc-gh-<env>-cc`) trusted only by GitHub jobs in the matching environment (OIDC subject `repo:<owner>/eurc-platform:environment:<env>`). Contributor on its own resource group plus an RBAC Administrator assignment conditioned to two data roles for managed identities.
- **Application:** `id-eurc-app-<env>-cc` , shared by both slots — Entra admin of the SQL server (migrations), Key Vault Secrets Officer (signing key), Storage Blob Data Contributor on *pos-receipts*.
- **Controls:** HTTPS only, TLS 1.2, FTP and basic-auth publishing off, SQL Entra-only, Storage shared keys and anonymous access off, Key Vault RBAC; in prod private endpoints, NSGs and delete locks.

6. Deployment procedure

Full steps, team exercises and troubleshooting are in `docs/RUNBOOK.md`; roles and review rules in `docs/TEAM.md`. From Azure Cloud Shell:

```
unzip eurc-p02-cicd-kit.zip && cd eurc-platform
nano config/eurc.env        # GITHUB_OWNER, DEV1, DEV2, TESTER, ALERT_EMAIL
gh auth login -s workflow
./setup.sh                  # 00 preflight → 01 Azure bootstrap → 02 GitHub team setup + first push
# team members accept the GitHub invitation; tester approves "prod" in Actions
./scripts/20-test-env.sh --integration
./scripts/03-harden-prod.sh
./scripts/02-setup-github.sh --protect-main
```

Script	Purpose
00-preflight.sh	tools, logins, Owner role, region policy, SQL Basic + App Service B1/S1 availability, providers, team settings
01-bootstrap-azure.sh	resource groups, OIDC deployment identities, scoped role assignments, budget
02-setup-github.sh	repository, collaborators, CODEOWNERS, environments with QA sign-off, variables, first push; --protect-main
03-harden-prod.sh	delete locks on prod SQL, Storage, Key Vault; checks private endpoints and public access
20-test-env.sh	21 / 21 / 29 conformance checks per environment + smoke (and integration)
30-monitor.sh	snapshot, watch, load + snapshot, test alert, DORA metrics
99-teardown.sh	removes locks, resource groups, soft-deleted vaults, budget; verifies

Local development needs no Azure: `npm run dev` in `app/` (in-memory API with a demo store) and in `web/` (Vite, proxies /api).

7. Verification performed before deployment

Check	Result	Detail
API tests (node:test)	**PASS**	31 / 31 — pricing, tokens, passwords, roles, checkout, oversell rollback, refunds, report, store isolation, health
API coverage	**PASS**	96.4 % lines (gate 80 %)
React tests (Vitest + Testing Library)	**PASS**	17 / 17 — money, Cart, CheckoutDialog, Login, whole App against a fake API
React coverage	**PASS**	91.0 % lines (gate 80 %)
ESLint (API, React with hooks rules)	**PASS**	0 problems
npm audit (runtime dependencies)	**PASS**	0 vulnerabilities (API and React)
CD build job, as written in cd.yml	**PASS**	API + React tests and build; one 44 MB zip with React in public/ and build.json
Packaged app, smoke suite	**PASS**	8 / 8 with the expected version; a wrong expected version is caught (6 / 8)
Integration suite (store trading day)	**PASS**	13 / 13, including oversell refusal, receipt in Blob, refund, isolation, clean-up
Load suite	**PASS**	20 s × 10 workers: 33 860 requests, 7 527 sales, p95 16 ms, 0 % errors (local)
Playwright E2E (tester-owned)	**PASS**	10 / 10 — 5 journeys on desktop and tablet against the packaged build
Bicep build + lint	**PASS**	main.bicep + 3 parameter files, 0 warnings (Bicep 0.48.1)
actionlint · ShellCheck	**PASS**	4 workflows + action · 11 scripts, 0 findings
Azure CLI calls	**PASS**	92 calls, 52 commands, 275 options checked against Azure CLI 2.91.0
deploy-infra and post-deploy gate (simulated Azure)	**PASS**	deploy / skip / force paths; gate passes at 0.25 %, rolls back at 7.5 %
Team setup (simulated GitHub)	**PASS**	collaborators invited, CODEOWNERS generated, prod reviewers = tester + lead with self-review blocked, kit pushed
Drawings	**PASS**	3 files × 2 A3 sheets render in draw.io 32.5.5

> **Not verifiable offline:** live Azure behaviour (quotas, policy, propagation) and GitHub behaviour that depends on real accounts (invitation acceptance, required reviewers on your plan). The runbook's troubleshooting table covers the likely failures.

8. Testing in Azure (procedure and evidence)

Fill in the result column from the first live run; evidence is in the GitHub run summaries, the `reports-*` artifacts (JUnit + Playwright HTML) and `reports/` in Cloud Shell.

#	Test	How	Expected	Result
T1	First CD run	push from 02-setup-github.sh	build → dev → test → (QA sign-off) → prod → release green	**PENDING**
T2	Environment conformance	20-test-env.sh	71 checks pass	**PENDING**
T3	POS end to end in the browser	Playwright in test stage	10 / 10 on desktop + tablet	**PENDING**
T4	Feature via PR (Dev 2 → Dev 1 review)	RUNBOOK exercise 1	CI green; schema 4 in all environments	**PENDING**
T5	Tester-owned test change	exercise 2	only the tester can approve tests/	**PENDING**
T6	CI blocks broken change	exercise 3	web check red; merge blocked	**PENDING**
T7	Bad release stopped in test	exercise 4	integration + E2E fail; no prod approval requested	**PENDING**
T8	QA sign-off and rollback	exercise 5	tester approves; rollback in ~1–2 min	**PENDING**
T9	Prod data tier private	part of T2	Key Vault 403, Storage 403 from the internet	**PENDING**
T10	Alert delivery · DORA	30-monitor.sh --test-alert / --dora	e-mail received · metrics printed	**PENDING**

9. Monitoring and operations

Signal	Condition	Severity
Availability test (3 locations, every 10 min)	/health fails from 2 or more locations	0
HTTP 5xx	more than 5 in 5 minutes	1
Health check	below 100 %	1
Response time	average above 2 s	2
Plan CPU · SQL DTU	above 80 %	2
Slot swap	every release or rollback	information

The API and its dependencies (SQL, Blob, Key Vault) report to Application Insights through OpenTelemetry, tagged with version, environment and slot. `30-monitor.sh` shows per environment and slot the running

version, health, requests, failures, p95 and exceptions, plus fired alerts and cost; `--dora` reports deployment frequency, lead time, change failure rate and time to restore.

10. Cost

Item	Quantity	USD per day (approx.)
App Service plans	2 × B1, 1 × S1	3.30
Azure SQL Basic	3	0.50
Private endpoints and private DNS zones	3 + 3	0.77
Availability test	432 runs/day	0.22
Storage, Key Vault, Log Analytics, App Insights	small volumes	0.10
Total		**≈ 4.90**

GitHub: free for a public repository (Actions minutes, environments, required reviewers). A monthly budget (USD 60) e-mails at 50 % / 80 % actual and 100 % forecast; `99-teardown.sh` removes everything.

11. Risks, limitations and decisions

Topic	Decision / risk	Mitigation
Card payments	No payment-gateway integration; only the terminal approval code is recorded	Keeps the lab out of PCI scope; a gateway can be added later
Small team approvals	With 3 people, absence blocks merges or releases	Lead is backup prod approver; owner can review any area
Private repo on GitHub Free	Environment reviewers and rulesets need a paid plan	Use a public repo or GitHub Pro / Team (Student Pack)
S1 quota on student subscriptions	Some offers have no Standard quota	Preflight checks; runbook gives a B1-without-slot fallback
App identity is SQL Entra admin	Simpler migrations; broader than needed	Split migration and runtime identities for real production
Shared database during swap	Old and new versions run together	Expand/contract rule enforced by a unit test
Test data in dev/test	Integration, load and E2E create stores	Every suite deletes its stores; prod gate is read-only

Appendix A. Repository inventory

File	File
`.github/CODEOWNERS`	`.github/ISSUE_TEMPLATE/bug_report.yml`
`.github/ISSUE_TEMPLATE/feature_request.yml`	`.github/actions/deploy-infra/action.yml`
`.github/pull_request_template.md`	`.github/scripts/post-deploy-gate.sh`
`.github/workflows/_deploy-env.yml`	`.github/workflows/cd.yml`
`.github/workflows/ci.yml`	`.github/workflows/rollback.yml`
`.gitignore`	`.shellcheckrc`
`README.md`	`app/eslint.config.js`
`app/package-lock.json`	`app/package.json`
`app/src/adapters/azure.js`	`app/src/adapters/memory.js`
`app/src/app.js`	`app/src/config.js`
`app/src/domain/auth.js`	`app/src/domain/pricing.js`
`app/src/migrations.js`	`app/src/seed.js`
`app/src/server.js`	`app/src/telemetry.js`
`app/test/app.test.js`	`app/test/domain.test.js`
`app/test/helpers.js`	`app/test/migrations.test.js`
`config/eurc.env`	`docs/RUNBOOK.md`
`docs/TEAM.md`	`docs/eURC_HandsOn_Project02_Report.pdf`
`docs/eURC_P02_Application_Architecture_A3.drawio`	`docs/eURC_P02_Application_Architecture_A3.pdf`
`docs/eURC_P02_Azure_Architecture_A3.drawio`	`docs/eURC_P02_Azure_Architecture_A3.pdf`
`docs/eURC_P02_DevOps_Architecture_A3.drawio`	`docs/eURC_P02_DevOps_Architecture_A3.pdf`
`infra/bicepconfig.json`	`infra/env/dev.bicepparam`
`infra/env/prod.bicepparam`	`infra/env/test.bicepparam`
`infra/main.bicep`	`infra/modules/alerts.bicep`
`infra/modules/data.bicep`	`infra/modules/monitoring.bicep`
`infra/modules/network.bicep`	`infra/modules/web.bicep`
`scripts/00-preflight.sh`	`scripts/01-bootstrap-azure.sh`

File	File
scripts/02-setup-github.sh	scripts/03-harden-prod.sh
scripts/20-test-env.sh	scripts/30-monitor.sh
scripts/99-teardown.sh	scripts/law-query.sh
scripts/lib.sh	setup.sh
tests/e2e/pos.spec.mjs	tests/integration.mjs
tests/lib.mjs	tests/load.mjs
tests/package-lock.json	tests/package.json
tests/playwright.config.mjs	tests/pos-client.mjs
tests/smoke.mjs	web/eslint.config.js
web/index.html	web/package-lock.json
web/package.json	web/public/favicon.svg
web/src/App.jsx	web/src/App.test.jsx
web/src/api.js	web/src/components/Cart.jsx
web/src/components/Cart.test.jsx	web/src/components/CheckoutDialog.jsx
web/src/components/CheckoutDialog.test.jsx	web/src/components/Login.jsx
web/src/components/Login.test.jsx	web/src/components/Products.jsx
web/src/components/Receipt.jsx	web/src/components/Register.jsx
web/src/components/Report.jsx	web/src/components/Sales.jsx
web/src/components/Team.jsx	web/src/lib/dates.js
web/src/lib/money.js	web/src/lib/money.test.js
web/src/lib/useLoader.js	web/src/main.jsx
web/src/styles.css	web/src/test-setup.js
web/vite.config.js	